

Examen PF2 2023–2024 — Documents autorisés — durée 1h30

L'utilisation des fonctions de la librairie standard OCaml est autorisée **et même attendue**. Cf. dernière page pour un rappel de quelques fonctions.

1 GADT : indexation d'une liste

Dans cette section, on étudie la question de l'indexation sûre d'une liste de taille n . Dans tout ce partiel, on considère qu'une liste de taille n est indexée de 0 à $n-1$.

On rappelle les définitions de types suivantes :

```
(* entiers de Peano au niveau des types *)
type zero = ...
type _ succ = ...
(* type des listes de taille n *)
type (_, _) nlist =
  | Nil : (_, zero) nlist
  | Cons : 'a * ('a, 'n) nlist -> ('a, 'n succ) nlist
```

Exemple d'utilisation :

```
# let l1 = Cons (7, Cons (8, Cons (9, Nil)));;
val l1 : (int, zero succ succ succ) nlist = Cons (7, Cons (8, Cons (9, Nil)))
```

1.1 Fonctions partielles

■ **Exercice 1:** Définir une fonction `nth` : `('a, 'n) nlist -> int -> 'a` telle que `nth xs i` renvoie l'élément d'indice i dans la liste `xs`. Tout cas d'erreur sera traité au moyen de la fonction `failwith`.

■ **Exercice 2:** Définir une fonction `insert` : `'a -> int -> ('a, 'n) nlist -> ('a, 'n succ) nlist` telle que `insert a i xs` insère `a` à la position i dans `xs`. Si l'indice i est égal à la longueur de la liste, l'élément `a` est ajouté en bout de liste. Tout cas d'erreur sera traité au moyen de la fonction `failwith`.

1.2 Fonctions totales

On souhaite maintenant écrire une version totale `nth2` de la fonction `nth` (sans cas d'erreur). Pour ce faire, il faut garantir que l'indice passé en paramètre est un entier naturel strictement inférieur à la longueur de la liste. On définit donc le GADT suivant :

```
type _ below =
  | Zero : 'n succ below
  | Succ : 'n below -> 'n succ below
```

Le type `n below` représente l'ensemble $[n] = \{i \in \mathbb{N} \mid 0 \leq i < n\}$. On observe que, grâce à la présence de la variable de type présente dans le type de `Zero`, une donnée peut appartenir à plusieurs types :

```
# (Zero : zero succ below);;      (* 0 ∈ [1] *)
- : zero succ below = Zero
# (Zero : zero succ succ below);; (* 0 ∈ [2] *)
- : zero succ succ below = Zero
# (Zero : zero below);;           (* 0 ∉ [0] *)
Error: This expression has type 'a succ below
      but an expression was expected of type zero below
      Type 'a succ is not compatible with type zero
# (Succ Zero : zero succ succ below);; (* 1 ∈ [2] *)
- : zero succ succ below = Succ Zero
# (Succ Zero : zero succ below);;    (* 1 ∉ [1] *)
Error: This expression has type 'a succ below
      but an expression was expected of type zero below
      Type 'a succ is not compatible with type zero
```

Exemple d'utilisation attendues de la fonction `nth2` :

```
# nth2 11 Zero;;
- : int = 7
# nth2 11 (Succ (Succ Zero));;
- : int = 9
# nth2 11 (Succ (Succ (Succ Zero)));;
Error: This expression has type 'a succ below
      but an expression was expected of type zero below
      Type 'a succ is not compatible with type zero
```

■ **Exercice 3:** Donner le type de la fonction `nth2`.

■ **Exercice 4:** Définir la fonction `nth2`.

On souhaite maintenant écrire une fonction totale `insert2` sur le même principe qu'`insert`. On reposera sur le type `_ below` pour les indices afin d'éviter la présence de cas d'erreur.

■ **Exercice 5:** Donner le type de `insert2`.

■ **Exercice 6:** Définir la fonction `insert2`.

2 Monade non-déterministe : résolution de problème

Le groupe de K-pop Blackpink, composé de Lisa, Rosé, Jennie et Jisoo, doit donner un concert dans 17 minutes au gala de l'ENSEEIH7. Toutefois, elles doivent traverser le canal pour rejoindre le gala (au départ, elles se situent toutes du côté opposé à l'ENSEEIH7) mais il fait nuit et elles ne disposent que d'une lampe de poche pour éclairer leur chemin. Elles ne peuvent traverser que par groupe de 1 ou 2 membres, et forcément avec la lampe de poche (la lampe de poche ne peut pas être jetée bien sûr). Chaque membre du groupe marche à une vitesse différente et un groupe de 2 doit caler sa vitesse sur celle de la membre la plus lente. Écrire un programme qui détermine une séquence de traversées permettant de rejoindre le gala dans les temps et faire ainsi vibrer les foules estudiantines.

Les caractéristiques du groupe Blackpink sont supposées données dans un module dont la signature est la suivante :

```
module Blackpink : sig
  type member = Lisa | Rose | Jennie | Jisoo
  (* 'compare' implémente un ordre total sur 'member'.
   'compare m1 m2' renvoie 0 si 'm1' = 'm2', -1 si 'm1' < 'm2', 1 sinon. *)
  val compare : member -> member -> int
  (* renvoie la vitesse, positive, de chaque membre. *)
  val speed : member -> int
end
```

Bien que cette connaissance ne soit pas formellement nécessaire pour résoudre le problème, voici la définition de `speed` :

```
let speed m = match m with | Lisa -> 1 | Rose -> 2 | Jennie -> 5 | Jisoo -> 10
```

Dans cette section, nous allons nous intéresser à la recherche de solutions à ce problème. Nous emploierons la monade (additive) non-déterministe vue en cours, dont la signature est rappelée ci-après :

```
module NDET : sig
  type 'a t = 'a list
  val map : ('a -> 'b) -> 'a t -> 'b t
  val return : 'a -> 'a t
  val ( >=> ) : 'a t -> ('a -> 'b t) -> 'b t (* bind *)
  val zero : 'a t
  val ( ++ ) : 'a t -> 'a t -> 'a t
end
```

L'algorithme de recherche a déjà été vu en cours (et, en version non-monadique, en PF1) ; en voici l'implémentation. Le but des exercices de cette section sera d'écrire les fonctions `neighbors` et `ok`, pour *in fine* retourner tous les chemins possibles, s'il y en a.

```
type pos = ... (* type abordé plus bas *)
type chemin = pos list
(* search : (pos -> pos NDET.t) -> (pos -> bool) -> pos -> chemin NDET.t *)
let rec search neighbors ok pos =
  if ok pos then return [ pos ]
```

```
else
  neighbors pos >>=
  search neighbors ok >>= fun chemin ->
  return (pos :: chemin)
```

On définit divers types en suivant.

- Le type `side = member NDET.t` représente l'ensemble des membres présents sur une rive;
- Le type `pos = side * side * bool * int` représente une position dans le graphe de recherche. Une position est donc de la forme `(side_from, side_to, side_from_is_far, time_left)` où :
 - `side_from` contient les membres sur la rive d'où aura lieu la prochaine traversée (s'il y en a une);
 - `side_to` représente les membres sur l'autre rive;
 - `side_from_is_far` vaut `true` si `side_from` est la rive opposée à l'ENSEEIH2, `false` si c'est le contraire;
 - `time_left` représente le temps encore disponible.
- Par exemple, la position initiale du problème est `([ Lisa; Rose; Jennie; Jisoo ], [], true, 17)`.
- Le type `chemin = pos list` représente enfin un chemin dans le graphe de recherche.

À toutes fins utiles, voici les deux solutions possibles à ce problème, en partant de l'état initial :

```
(* solution 1 *)
->: [Lisa, Rose]
<-: [Lisa]
->: [Jennie, Jisoo]
<-: [Rose]
->: [Lisa, Rose]
```

```
(* solution 2 *)
->: [Lisa, Rose]
<-: [Rose]
->: [Jennie, Jisoo]
<-: [Lisa]
->: [Lisa, Rose]
```

■ **Exercice 7:** Écrire la fonction `time_to_cross : member list -> int` qui calcule le temps nécessaire à une liste de membres pour traverser le pont.

```
# time_to_cross [];
- : int = 0
# time_to_cross [Lisa];
- : int = 1
# time_to_cross [Lisa; Rose];
- : int = 2
```

■ **Exercice 8:** Écrire la fonction `remove_list : 'a list -> 'a list -> 'a list` telle que `remove_list xs ys` retire les éléments de `xs` de la liste `ys`.

```
# remove_list [Lisa; Rose] [];
- : member list = []
# remove_list [Lisa; Rose] [Lisa];
- : member list = []
# remove_list [Lisa; Rose] [Lisa; Rose; Jennie];
- : member list = [Jennie]
# remove_list [] [Lisa; Rose; Jennie];
- : member list = [Lisa; Rose; Jennie]
```

■ **Exercice 9:** Écrire la fonction `select_people : side -> member list NDET.t` telle que `select_people rive` retourne l'ensemble des groupes (`member list`) de 1 ou 2 membres possibles sur cette rive. Indice : la fonction `compare` permettra (1) de déterminer si deux membres sélectionnées sont la même personne, et (2) de ne retourner un groupe de 2 que si le premier membre est (p. ex.) inférieur au second selon `compare` (ceci permettra d'éviter de multiplier les solutions équivalentes, puisque un groupe `[a;b]` est équivalent à un groupe `[b;a]`).

■ **Exercice 10:** Écrire la fonction `neighbors : pos -> pos NDET.t` qui renvoie l'ensemble des positions suivantes possibles à partir d'une position passée en paramètre, en utilisant toutes les fonctions écrites précédemment. Étant donnée une position suivante renvoyée, on assurera que l'union des membres des deux rives est égale à l'ensemble des membres du groupe.

Indice : pour trouver un voisin d'une position donnée, il faut déterminer une ou des membres qui peuvent traverser et s'assurer qu'il reste assez de temps pour qu'elle(s) traverse(nt). Si c'est le cas, la nouvelle position trouvée diffère de la première par le transfert de membres effectué, le temps restant, et surtout par le fait que la rive de départ de l'un est la rive d'arrivée de l'autre, et vice-versa (puisque la lampe change de rive à chaque fois).

■ **Exercice 11:** Écrire la fonction `ok : pos -> bool` qui renvoie vrai si la position passée en paramètre représente un état de succès de la recherche. Indices : on peut s'appuyer sur le booléen et sur le fait que si une membre n'est pas sur une rive, elle est sur l'autre.

Annexe : extrait du module List

```
val tl : 'a list -> 'a list
(** Return the given list without its first element.
  @raise Failure if the list is empty.
  *)
val mem : 'a -> 'a list -> bool
(** [mem a set] is true if and only if [a] is equal
  to an element of [set].
  *)
val iter : ('a -> unit) -> 'a list -> unit
(** [iter f [a1; ...; an]] applies function [f] in turn to
  [[a1; ...; an]]. It is equivalent to
  [f a1; f a2; ...; f an].
  *)
val map : ('a -> 'b) -> 'a list -> 'b list
(** [map f [a1; ...; an]] applies function [f] to [a1, ..., an],
  and builds the list [[f a1; ...; f an]]
  with the results returned by [f].
  *)
val fold_left : ('acc -> 'a -> 'acc) -> 'acc -> 'a list -> 'acc
(** [fold_left f init [b1; ...; bn]] is
  [f (... (f (f init b1) b2) ...) bn].
  *)
val fold_right : ('a -> 'acc -> 'acc) -> 'a list -> 'acc -> 'acc
(** [fold_right f [a1; ...; an] init] is
  [f a1 (f a2 (... (f an init) ...))]. Not tail-recursive.
  *)
val filter : ('a -> bool) -> 'a list -> 'a list
(** [filter f l] returns all the elements of the list [l]
  that satisfy the predicate [f]. The order of the elements
  in the input list is preserved.
  *)
```